

Hệ thống điều khiển tránh vật cản cho Micro UAV sử dụng FastViT và Perception-Aware MPPI

Hoàng Khánh Đồng
Sinh viên
Email: hkd2k4@gmail.com

Hoàng Công Phát
Mentor
VHT (Viettel High-Tech)
Email: mentor@viettel.com.vn

I. GIỚI THIỆU CHUNG

Ngày nay, phát hiện và tránh vật cản là một trong những chức năng cốt lõi của các hệ thống tự hành, đặc biệt trong các ứng dụng như Unmanned Aerial Vehicles (UAVs) và robot di động. Chức năng này đề cập đến khả năng của hệ thống trong việc nhận biết các chướng ngại vật trong môi trường và xây dựng quỹ đạo an toàn nhằm tránh va chạm.

Tuy nhiên, việc triển khai chức năng này trên các UAV kích thước nhỏ (Micro/Nano UAVs) bay ở tốc độ cao trong môi trường chật hẹp gặp phải rào cản lớn về phần cứng. Các UAV này không thể mang theo các cảm biến nặng như LiDAR hay GPU công kênh, khiến việc chạy các mô hình AI dự đoán chiều sâu (Depth Estimation) thường xuyên bị thất cổ chai về dung lượng VRAM và độ trễ (latency). Bên cạnh đó, các thuật toán quy hoạch quỹ đạo tối ưu như Model Predictive Path Integral (MPPI) truyền thống [1] lại đòi hỏi khả năng tính toán CPU khổng lồ và thường mắc phải lỗi kẹt cục bộ (Local Minimum) khi đối mặt với vật cản trực diện.

Để giải quyết các thách thức trên, báo cáo này trình bày một kiến trúc nhận thức và điều khiển toàn trình được tối ưu hóa cho thiết bị biên. Thứ nhất, hệ thống sử dụng mạng nơ-ron hạng nhẹ FastViT để dự đoán bản đồ độ sâu từ ảnh đơn sắc. Thứ hai, toàn bộ luồng quy hoạch quỹ đạo được xây dựng lại bằng thư viện JAX [2] nhằm tận dụng khả năng tính toán ma trận song song trên GPU. Đặc biệt, hệ thống áp dụng một hàm chi phí (Cost Function) cải tiến tích hợp *Alignment Penalty* và cơ chế dập quán tính (*Warm-start Decay*), giúp phá vỡ tính đối xứng của thuật toán, hạn chế hiện tượng kẹt tiến-lùi, hỗ trợ UAV lách qua chướng ngại vật với tần số phản xạ phù hợp.

II. NỘI DUNG VÀ PHƯƠNG PHÁP

A. Tổng quan hệ thống

Như được minh họa trong Hình 1, quy trình xử lý của hệ thống diễn ra theo một luồng liên mạch (pipeline) từ cảm biến cho đến thực thi. Bắt đầu từ thiết bị **RGB Camera**, hệ thống thu nhận **RGB Image** từ góc nhìn thứ nhất của UAV. Bức ảnh này sau đó được truyền vào cụm mô hình học sâu bao gồm **FastViT-T12** [3] kết hợp với **Custom DPT** [4] để nội suy và dự đoán ra bản đồ độ sâu (**Depth Image**).

Thay vì tính toán trực tiếp trên toàn bộ ảnh độ sâu, hệ thống tiến hành trích xuất và biến đổi ảnh này thành một đám mây điểm thưa thớt (**Sparse Point Cloud**) nhằm tối ưu hóa bộ nhớ và tăng tốc độ xử lý. Đám mây điểm này cuối cùng được đẩy

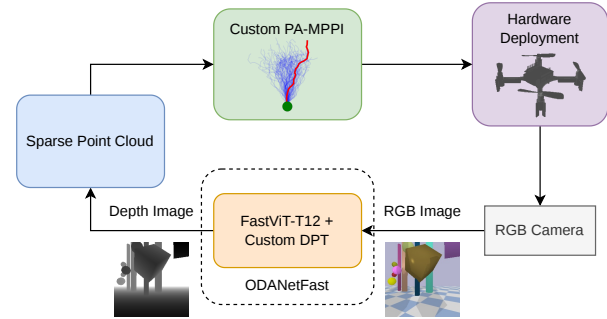


Fig. 1. Kiến trúc tổng quan của hệ thống điều khiển tránh vật cản UAV.

vào khối điều khiển **Custom PA-MPPI** để đánh giá chi phí va chạm và quy hoạch quỹ đạo an toàn. Tín hiệu vận tốc tối ưu sinh ra từ bộ điều khiển sẽ được nạp trực tiếp xuống UAV trong bước **Hardware Deployment**.

B. Dữ liệu huấn luyện và tiền xử lý

Để đảm bảo khả năng tổng quát hóa (Domain Generalization) của mạng nơ-ron từ môi trường mô phỏng sang môi trường thực tế, hệ thống áp dụng chiến lược học dữ liệu hỗn hợp (Mixed Dataset Distillation). Quá trình chuẩn bị và tiền xử lý dữ liệu được chia làm các bước sau:

Sinh dữ liệu mô phỏng (Synthetic Data Generation): Dữ liệu mô phỏng được tạo ra tự động thông qua engine vật lý PyBullet [5]. Các kịch bản bay được thiết kế ngẫu nhiên hóa (Domain Randomization) bao gồm: các cột chướng ngại vật hình trụ và các khối cầu lơ lửng đa dạng. Ở mỗi tập kịch bản, hệ thống lấy mẫu tọa độ không gian và cấu hình góc nhìn Camera (Pitch, Yaw) ngẫu nhiên để mô phỏng sự chao đảo của UAV khi bay tốc độ cao. Dữ liệu thu thập bao gồm ảnh RGB kích thước 224×224 và bản đồ độ sâu tuyệt đối (Metric Depth). Đặc biệt, tham số mặt phẳng cắt xa (Far Clipping Plane) của camera mô phỏng được thiết lập cố định ở mức 10 mét.

Dữ liệu thực tế (ODADataset) và Pseudo-depth: Bên cạnh dữ liệu mô phỏng, tập dữ liệu còn được kết hợp với bộ dữ liệu thực tế mang tên **ODADataset** (Obstacle Detection and Avoidance Dataset) được trích xuất từ các video ghi hình chuyển bay (.avi). Để cung cấp nhãn chiều sâu (label) cho các ảnh thực tế này, hệ thống sử dụng mô hình nền tảng **Depth**

Anything V2 (DA2) [6] - phiên bản Small (khoảng 25 triệu tham số) nhằm trích xuất bản đồ độ sâu giả (Pseudo-depth). Sự pha trộn này ép buộc mạng ODANetFast phải học cách tìm ra điểm cân bằng toán học giữa khoảng cách không gian tuyệt đối của PyBullet và khả năng nội suy mượt mà, chống nhiễu của mô hình DA2, từ đó tăng cường độ mạnh mẽ (robustness) khi bay thực tế.

Tiền xử lý và tăng cường dữ liệu (Data Augmentation): Trong quá trình nạp dữ liệu (Dataloader), các kỹ thuật tăng cường dữ liệu được áp dụng thời gian thực (On-the-fly) bao gồm: thay đổi màu sắc (Color Jittering) và lật ảnh ngẫu nhiên (Random Horizontal Flip) nhằm hạn chế Overfitting. Ảnh đầu vào được chuẩn hóa (Normalization) theo phân phối chuẩn của ImageNet với trung bình (mean) là $[0.485, 0.456, 0.406]$ và độ lệch chuẩn (std) là $[0.229, 0.224, 0.225]$.

Đối với bản đồ độ sâu (Ground Truth), hàm mất mát thực hiện thao tác cắt cụt (Clamping) trần tại ngưỡng 10.0 mét. Kỹ thuật này đóng vai trò như một biên độ an toàn (Safety Margin), giúp bộ điều khiển hạ nguồn (MPPI) phân biệt rõ ràng không gian an toàn ("bầu trời") và tránh được các hành vi né tránh chướng ngại vật ảo (Phantom Obstacles) do sai số hồi quy của mạng gây ra.

C. Mạng nơ-ron dự đoán độ sâu (FastViT-T12 + Custom DPT)

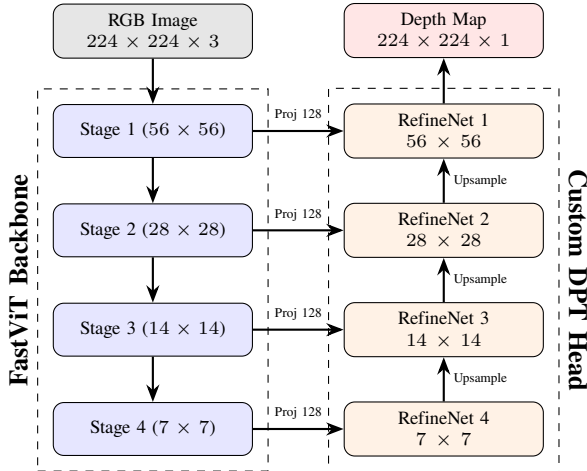


Fig. 2. Kiến trúc mạng nơ-ron ODANetFast (FastViT-T12 + Custom DPT).

Mạng nơ-ron nhận thức của hệ thống (ODANetFast) đóng vai trò tiếp nhận ảnh FPV RGB dạng phẳng và nội suy ra bản đồ độ sâu của môi trường. Điểm đặc biệt của cấu trúc này là thiết kế lược giản (stripped-down), loại bỏ hoàn toàn các module dựng lưới (Voxelization) hay tạo đám mây điểm nội bộ nhằm giảm tải tài nguyên tính toán (VRAM). Việc xử lý hình học 3D được hoãn lại (deferred) cho khối JAX phía sau. Kiến trúc của ODANetFast bao gồm hai phần chính:

1. Trích xuất đặc trưng với FastViT Backbone:

Mạng sử dụng kiến trúc FastViT-T12 [3] (khởi tạo với trọng số Pre-trained trên ImageNet) làm xương sống. Ảnh đầu vào kích thước 224×224 pixel khi truyền qua mạng sẽ được trích xuất thành 4 bản đồ đặc trưng đa cấp độ (multi-scale feature maps) tương ứng với 4 giai đoạn (stages). Với các hệ số thu

gọn (reduction factors) lần lượt là 4, 8, 16 và 32, mô hình tạo ra các khối đặc trưng có độ phân giải không gian giảm dần (56×56 , 28×28 , 14×14 và 7×7), cho phép nắm bắt từ các chi tiết cạnh, góc nhỏ nhất cho đến ngữ cảnh vật cản tổng thể.

2. Giải mã không gian với Custom DPT Head:

Để khôi phục lại không gian 3D, hệ thống sử dụng một bộ giải mã Dense Prediction Transformer (DPT) [4] tùy chỉnh. Đầu tiên, 4 bản đồ đặc trưng từ FastViT đi qua các lớp tích chập đồng bộ hóa (Projection layers) để đưa số lượng kênh (channels) về mức chung là 128 kênh. Tiếp đó, quá trình dung hợp (Feature Fusion) diễn ra thông qua chuỗi 4 khối mạng RefineNet nối tiếp nhau. Mỗi khối RefineNet bao gồm các đơn vị tích chập phần dư (ResidualConvUnit) kết hợp với thuật toán nội suy song tuyến tính (Bilinear Interpolation) để khuếch đại (upsample) và hòa trộn thông tin từ dạng ngữ nghĩa chiều sâu về dạng chi tiết hình ảnh.

Sau khi đi qua khối RefineNet cuối cùng, bản đồ đặc trưng được phóng to trả về đúng kích thước gốc (224×224) và qua lớp tích chập đầu ra để nén thành 1 kênh độ sâu duy nhất. Cuối cùng, hàm kích hoạt Softplus được áp dụng lên bản đồ thô (raw map) này, đảm bảo các giá trị độ sâu dự đoán luôn tròn trịa và nhận giá trị dương, phản ánh đúng tính chất khoảng cách vật lý của môi trường thực.

3. Thông số mô hình và Dữ liệu huấn luyện:

Mô hình ODANetFast được thiết kế nhỏ gọn để phù hợp triển khai trên thiết bị biên. Cụ thể, phần xương sống **FastViT-T12** chiếm khoảng 6.38 triệu tham số, và phần đầu giải mã **Custom DPT** chiếm 2.94 triệu tham số. Tổng cộng, toàn bộ mạng có khoảng **9.32 triệu tham số** ($\sim 9.3M$ parameters). Để huấn luyện mô hình, một bộ dữ liệu hỗn hợp đa dạng đã được tổng hợp với tổng cộng 78,861 mẫu huấn luyện (mixed training samples). Trong đó, bộ dữ liệu mô phỏng từ PyBullet đóng góp 20,000 mẫu, kết hợp với các chuỗi ảnh thực tế được chia thành 73,176 chuỗi con (subsequences). Quá trình phân tách dữ liệu được thực hiện cẩn thận với tập huấn luyện (Train split) bao gồm 58,861 mẫu (952 chuỗi) và tập kiểm định (Val split) gồm 14,315 mẫu (238 chuỗi). Sự đa dạng và dung lượng lớn của tập dữ liệu huấn luyện giúp mô hình khái quát hóa tốt, thu hẹp hiệu quả khoảng cách giữa mô phỏng và thực tế (Sim-to-Real gap).

4. Hàm suy hao (Loss Functions) và Chỉ số đánh giá:

Trong quá trình huấn luyện và chứng cất tri thức (Knowledge Distillation) từ mô hình giáo viên (Depth Anything V2) sang mô hình học sinh (ODANetFast), hệ thống sử dụng hàm suy hao kết hợp (Hybrid Metric Depth Loss):

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{SILog}} + w_{L1} \cdot \mathcal{L}_{L1} \quad (1)$$

Trong đó, $\mathcal{L}_{L1}$ là hàm sai số tuyệt đối (L1 Loss) dùng để bảo toàn thang đo vật lý (metric scale), với trọng số $w_{L1} = 0.1$. $\mathcal{L}_{\text{SILog}}$ là hàm suy hao logarit bất biến tỷ lệ (Scale-Invariant Logarithmic Loss) tập trung học các đặc trưng hình học bất kể thang đo, được định nghĩa như sau:

$$D_i = \log(d_i) - \log(d_i^*) \quad (2)$$

$$\mathcal{L}_{\text{SILog}} = \lambda \sqrt{\frac{1}{n} \sum_{i=1}^n D_i^2 - \frac{\alpha}{n^2} \left(\sum_{i=1}^n D_i \right)^2} \quad (3)$$

với d_i là độ sâu dự đoán, d_i^* là độ sâu tham chiếu thực (Ground Truth / Pseudo-depth), $\lambda = 1.0$ là hệ số tỷ lệ và $\alpha = 0.5$ là hệ số phạt phương sai. Quá trình tính toán chỉ diễn ra trên các điểm ảnh có giá trị nằm trong ngưỡng hợp lệ ($0 < d_i^* \leq 10.0$).

Để đánh giá định lượng chất lượng bản đồ độ sâu trong giai đoạn kiểm định (Validation), báo cáo sử dụng chỉ số sai số tương đối tuyệt đối (Absolute Relative Error - AbsRel). Do mô hình giáo viên đôi khi thiếu tính đồng nhất về mặt thang đo, chỉ số này được tích hợp thêm hệ số chuẩn hóa theo trung vị (Median Scaling) nhằm bù trừ sai lệch tỷ lệ:

$$s = \frac{\text{median}(d^*)}{\text{median}(d)} \quad (4)$$

$$\text{AbsRel} = \frac{1}{n} \sum_{i=1}^n \frac{|s \cdot d_i - d_i^*|}{d_i^*} \quad (5)$$

D. Khối trích xuất đám mây điểm (Sparse Point Cloud)

Bản đồ độ sâu sau khi được suy luận từ ODANetFast mang khối lượng dữ liệu rất lớn ($224 \times 224 \approx 50,000$ điểm ảnh). Nếu nạp toàn bộ số điểm này vào bộ điều khiển MPPI, hệ thống sẽ rơi vào trạng thái thắt cổ chai tính toán (computational bottleneck). Do đó, hệ thống sử dụng một khối trích xuất đám mây điểm tính (Rolling Sparse Map) được lập trình hoàn toàn bằng JAX nhằm khai thác tối đa sức mạnh của GPU.

Quy trình trích xuất bao gồm các bước toán học như sau:

1. Phép chiếu ngược (Back-projection) và Lấy mẫu thưa:

Từ bản đồ độ sâu $D(u, v)$ tại tọa độ pixel (u, v) , không gian 3D tương đối trong hệ quy chiếu camera được khôi phục thông qua các thông số nội sinh (Intrinsic Parameters) bao gồm tiêu cự (f_x, f_y) và tâm quang học (c_x, c_y):

$$z_{cam} = D(u, v) \quad (6)$$

$$x_{cam} = (u - c_x) \frac{z_{cam}}{f_x} \quad (7)$$

$$y_{cam} = (v - c_y) \frac{z_{cam}}{f_y} \quad (8)$$

Để tối ưu, thuật toán không chuyển đổi toàn bộ điểm ảnh. Thay vào đó, bộ lọc sẽ loại bỏ các điểm nhiễu (quá gần hoặc vượt quá ngưỡng cắt max_depth) và chỉ lấy mẫu ngẫu nhiên $N = 2048$ điểm hợp lệ để tạo thành đám mây điểm cục bộ $\mathbf{P}_{cam} = [x_{cam}, y_{cam}, z_{cam}]^T$.

2. Hiệu chỉnh góc ngẩng (Pitch Correction):

Do camera FPV thường được gắn nghiêng một góc θ (Pitch) so với thân máy bay (Body Frame) để nhìn rõ vật cản khi UAV lao về phía trước, tọa độ của các điểm cần được xoay lại cho đồng nhất với hệ trục của UAV:

$$\begin{bmatrix} x_{body} \\ y_{body} \\ z_{body} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \sin \theta & \cos \theta \\ 0 & -\cos \theta & \sin \theta \end{bmatrix} \begin{bmatrix} x_{cam} \\ y_{cam} \\ z_{cam} \end{bmatrix} \quad (9)$$

Lưu ý: Ma trận quay này được điều chỉnh dành riêng cho cấu hình trục tọa độ camera đặc thù của engine mô phỏng, trong đó trục +Z hướng về phía trước và trục +Y hướng xuống.

3. Chuyển đổi toàn cục và Lọc Voxel (Voxel Grid Filter):

Các điểm sau khi hiệu chỉnh sẽ được chuyển đổi tiếp từ hệ trục thân máy bay sang hệ quy chiếu toàn cục (World Frame) sử dụng ma trận xoay $\mathbf{R}_w$ và véc-tơ tịnh tiến $\mathbf{p}_w$ của UAV lấy từ hệ thống đo lường:

$$\mathbf{P}_{world} = \mathbf{R}_w \cdot \mathbf{P}_{body} + \mathbf{p}_w \quad (10)$$

Những điểm này sau đó được nạp vào một bộ đệm cuộn (Rolling Buffer) lưu trữ 100 khung hình gần nhất, đóng vai trò như một bản đồ cục bộ có khả năng ghi nhớ không gian chướng ngại vật xung quanh. Nhằm loại bỏ các điểm trùng lặp do quét chồng chéo giữa các frame, thuật toán áp dụng bộ lọc Voxel Grid bằng phép làm tròn với kích thước khối lưới $V_s = 0.15$ mét:

$$\mathbf{P}_{voxel} = \text{round} \left(\frac{\mathbf{P}_{world}}{V_s} \right) \cdot V_s \quad (11)$$

Cuối cùng, hàm `jnp.unique` được sử dụng để lọc ra một danh sách điểm độc nhất mang kích thước cố định (ví dụ: 10,000 điểm). Việc duy trì các tensor ở kích thước tĩnh (static shape) là chìa khóa then chốt giúp trình biên dịch JIT (Just-In-Time) của JAX biên dịch trước thành các hàm phần cứng tối ưu, qua đó giảm độ trễ thực thi xuống chỉ còn cỡ mili-giây.

E. Bộ điều khiển tránh vật cản (Custom PA-MPPI)

Thuật toán Model Predictive Path Integral (MPPI) nhận thức (Perception-Aware MPPI) là một kỹ thuật điều khiển tối ưu dự báo mạnh mẽ. Bản chất của thuật toán là lấy mẫu ngẫu nhiên N quỹ đạo trong tương lai, tính toán hàm chi phí $\mathcal{L}$ cho từng quỹ đạo, sau đó tổng hợp lại thông qua trung bình có trọng số (Softmax-weighted average) để đưa ra lệnh điều khiển tối ưu.

Mặc dù PA-MPPI nguyên bản giải quyết bài toán điều hướng không gian chưa biết [7], thuật toán này dựa trên mô hình động lực học 13 trạng thái phức tạp và phụ thuộc vào bản đồ lưới Voxel (Occupancy Grid) tốn kém bộ nhớ. Nhằm đáp ứng khả năng chạy thời gian thực trên các vi xử lý biên, báo cáo này trình bày một kiến trúc điều khiển **Custom PA-MPPI** được thiết kế lại từ PA-MPPI [7] và viết hoàn toàn bằng JAX. Các thay đổi bao gồm: (1) đơn giản hóa dynamics, (2) thay thế collision cost bằng soft-margin trên point cloud, (3) bổ sung heading cost và progress bất đối xứng để thay thế cơ chế two-phase switching, và (4) tinh gọn perception cost phù hợp sparse representation. Những thay đổi này nhằm hỗ trợ tính toán thời gian thực trong khi vẫn duy trì khả năng tránh vật cản và tiến về đích:

1. Động lực học lệnh vận tốc (Velocity-Command Dynamics):

Thay vì điều khiển lực đẩy (Thrust) và tốc độ góc (Body-rates) như nguyên bản, mô hình được chuyển đổi sang không gian trạng thái 7 chiều $\mathbf{x} = [p_x, p_y, p_z, v_x, v_y, v_z, \psi]^T$ và không gian lệnh 4 chiều $\mathbf{u} = [v_x^{\text{cmd}}, v_y^{\text{cmd}}, v_z^{\text{cmd}}, \psi^{\text{cmd}}]^T$. Một bộ lọc nhiễu trễ cấp một (First-order lag) được áp dụng để mô phỏng sự phản hồi thực tế của bộ điều khiển PID cấp thấp:

$$v_i^{k+1} = v_i^k + \frac{\Delta t}{\tau + \Delta t} (v_i^{\text{cmd}} - v_i^k) \quad (12)$$

với $\tau \approx 0.1s$ là hằng số thời gian đáp ứng và $\Delta t = 0.05s$.

2. Hàm chi phí va chạm liên tục (Continuous Soft-margin Collision):

Thuật toán PA-MPPI gốc [7] sử dụng hàm phạt va chạm nhị phân (Binary Indicator) dạng:

$$\ell_{\text{collision_orig}} = c_{\text{collision}} \cdot \mathbf{1}_{\{\mathcal{G}(\mathbf{p}) \neq 0\}}$$

Điều này gây khó khăn trong việc cung cấp độ dốc (Gradient) hướng dẫn khi lấy mẫu trên Point Cloud. Thay vào đó, hệ thống xây dựng một trường tiềm năng (Potential Field) kết hợp giữa hàng rào hàm mũ và lực đẩy mềm:

$$\ell_{\text{col}} = \sum_{k=0}^{H-1} c_{\text{col}} \exp\left(-\frac{d_k^{\min} - r_{\text{safe}}}{\sigma_{\text{col}}}\right) + \frac{c_{\text{rep}}}{d_k^{\min} + 0.5} \quad (13)$$

trong đó $d_k^{\min}$ là khoảng cách ngắn nhất đến đám mây điểm chướng ngại vật tại bước k . Hàm này giúp UAV lượn lách mượt mà qua các khe hẹp mà không bị dao động đột ngột.

3. Phạt hành trình bất đối xứng (Asymmetric Progress Cost):

Trong phase tiến về đích của PA-MPPI gốc [7] (khi goal nằm trong tầm nhìn trực tiếp), hàm hành trình có dạng đối xứng: $\ell_{\text{progress_orig}} = -c_{\text{progress}} \cdot \|\mathbf{p}_k - \mathbf{p}_{k+1}\|$ được sử dụng để khuyến khích UAV di chuyển nhanh theo đường thẳng đến đích. Tuy nhiên, thuật toán MPPI truyền thống thường rơi vào trạng thái kẹt (Local Minimum) hoặc lùi lại khi đối mặt trực diện với tường lớn. Để giải quyết, một hàm tiến/lùi bất đối xứng được đưa vào:

$$\ell_{\text{prog}} = \sum_{k=0}^{H-2} \begin{cases} -c_{\text{fwd}} \cdot \Delta_k & \text{nếu } \Delta_k > 0 \\ c_{\text{bwd}} \cdot |\Delta_k| & \text{nếu } \Delta_k \leq 0 \end{cases} \quad (14)$$

với $\Delta_k = d_k - d_{k+1}$ là mức độ tiến gần đích. Với trọng số lùi $c_{\text{bwd}} = 12.0$ lớn gấp 2.4 lần trọng số tiến $c_{\text{fwd}} = 5.0$, UAV bị phạt nặng nếu bay lùi, ép hệ thống phải rẽ trái sang hai bên để tìm đường thoát.

4. Đồng bộ hướng bay và Nhận thức (Velocity-to-Goal Heading & Perception):

PA-MPPI gốc [7] sử dụng ray-tracing trên occupancy grid để đánh giá vùng occupied và unknown nhằm khuyến khích khám phá. Tuy nhiên, với kiến trúc Rolling Sparse Point Cloud của hệ thống, việc duy trì occupancy grid 3 trạng thái đòi hỏi bộ nhớ đáng kể. Do đó, hàm chi phí nhận thức được tinh gọn, chỉ giữ lại thành phần Camera-to-Goal Alignment [7] để đảm bảo UAV hướng về phía đích:

$$\ell_{\text{perception}} = \sum_{k=0}^{H-1} c_{\text{PoI}} \cdot \left(1 - \langle \hat{\mathbf{x}}_{WB}, \hat{\mathbf{d}}_{\text{goal}} \rangle\right)^2 \cdot \mathbf{1}_{\{d_k > c_{\text{thresh}}\}} \quad (15)$$

Khả năng khám phá vùng chưa biết được bù đắp gián tiếp thông qua hàm đồng bộ hướng bay (Velocity-to-Goal Heading):

$$\ell_{\text{head}} = \sum_{k=0}^{H-1} c_{\text{head}} \left(1 - \langle \hat{\mathbf{v}}_k, \hat{\mathbf{d}}_{\text{goal}} \rangle\right) \cdot \mathbf{1}_{\{\|\mathbf{v}_k\| > v_{\text{thresh}}\}} \quad (16)$$

Khác với PA-MPPI gốc [7] sử dụng cơ chế chuyển phase hai giai đoạn (exploration khi goal bị che khuất / goal-reaching

khi trong tầm nhìn), Custom PA-MPPI tích hợp hàm chi phí heading và progress bất đối xứng để tự động điều tiết hành vi mà không cần chuyển mạch rõ ràng, qua đó giảm độ phức tạp tính toán.

5. Rào cản trần bay (Altitude Safety):

Bổ sung hàm ràng buộc không gian bay an toàn theo trục Z nhằm tránh hiện tượng sạt lún xuống sàn hoặc bay vọt lên quá cao:

$$\ell_{\text{floor}} = \sum_{k=0}^{H-1} c_{\text{floor}} \left[(z_{\min} - p_z^k)_+^2 + (p_z^k - z_{\max})_+^2 \right] \quad (17)$$

trong đó ký hiệu $(\cdot)_+$ đại diện cho hàm $\max(0, \cdot)$.

Cuối cùng, nhờ tận dụng khả năng vector hóa (Vmap) của XLA/JAX, bộ điều khiển có thể mô phỏng song song $N = 10,000$ quỹ đạo với chu kỳ $H = 30$ bước, tận dụng tối đa băng thông bộ nhớ GPU và đạt tần số phản xạ thời gian thực ổn định ở mức 50 Hz.

TABLE I
BẢNG TỔNG HỢP SO SÁNH PA-MPPI GỐC VÀ CUSTOM PA-MPPI

Thành phần	PA-MPPI gốc [7]	Custom PA-MPPI (Hệ thống)	Lý do thay đổi
Dynamics	13-state	7-state velocity-command	Edge deployment
Collision	Binary indicator	Soft-margin exp + repulsive	Point cloud, cần gradient
Perception	PoI + ray-tracing	PoI only	Không có occupancy grid
Progress	Đối xứng	Bất đối xứng fwd/bwd	Tránh local minimum
Phase Switch	2 phase (Exploration/Reach)	Không chuyển mạch	Giảm độ phức tạp tính toán
Altitude	Không giới hạn	Rào cản sàn/trần	Tránh hiện tượng floor-diving

III. THIẾT LẬP MÔ PHỎNG

A. Thông số phần cứng

Quá trình huấn luyện mô hình học sâu và mô phỏng hệ thống (Software-in-the-Loop) được thực hiện trên một máy tính cá nhân với cấu hình như sau:

- **CPU:** Intel® Core™ i5-11400 @ 2.60GHz
- **Bộ nhớ RAM:** 32 GB
- **GPU:** NVIDIA® GeForce® RTX 3060 (12 GB VRAM)

B. Cấu hình Drone và Bộ điều khiển PA-MPPI

Môi trường mô phỏng được xây dựng dựa trên engine vật lý **PyBullet** thông qua nền tảng `gym-pybullet-drones` [8]. Mô hình UAV được sử dụng là vi thiết bị bay quadrotor **Bitcraze Crazyflie 2.X (CF2X)** với các thông số vật lý và giới hạn bay được thiết lập gắt gao nhằm phản ánh chính xác động lực học thực tế. Các thông số phần cứng được trích xuất trực tiếp từ file cấu hình URDF (`cf2x.urdf`):

TABLE II
Thông số vật lý và thiết lập động lực học của UAV CF2X.

Physical Params		Dynamics & Limits	
Trọng lượng (m)	0.027 kg	$v_{\max}$	5.0 m/s
Chiều dài tay đòn (l)	39.7 mm	$a_{\max}$	5.0 m/s ²
I_{xx}, I_{yy}	1.4e-5	Góc nghiêng tối đa	30°
I_{zz}	2.17e-5	$f_{\text{sim}}, f_{\text{ctrl}}$	240 Hz
Collision r, h	6 cm, 2.5 cm	Thrust/Weight	2.25
k_f	3.16e-10	k_m	7.94e-12

Về phần **Custom PA-MPPI**, nhằm khai thác tối đa sức mạnh tính toán song song của GPU RTX 3060 thông qua trình biên dịch JAX, bộ điều khiển được thiết lập với bộ siêu tham số (hyperparameters) mang cường độ tính toán cao:

TABLE III
Bộ siêu tham số và Trọng số Hàm chi phí (Cost Weights) của Custom PA-MPPI.

MPPI param.		Cost definition constants			
N	10,000	c_{goal}	8.0	$c_{\text{collision}}$	80.0
H	30	$c_{\text{goal_term}}$	40.0	$c_{\text{repulsive}}$	0.8
Δt	0.05 s	c_{Pol}	10.0	c_{heading}	3.0
u_z limit	$[-1.0, 1.0]$ m/s	c_{thresh}	1.0	$c_{\text{prog_fwd}}$	5.0
σ_z	0.5	$c_{\text{vel_damp}}$	2.0	$c_{\text{prog_bwd}}$	12.0
f_{ctrl}	240 Hz	c_{safe}	50.0	R	diag(0.01, 0.01, 0.02, 0.15)
		safe_margin	0.45	R_{Δ}	diag(0.03, 0.03, 0.05, 0.05)

C. Thiết lập môi trường mô phỏng (Simulation Environment)

Môi trường thử nghiệm được thiết kế dưới dạng một đường đua thẳng (straight racing track) dọc theo trục Y, bao gồm các thành phần tạo ra thử thách né tránh phức tạp cho UAV trong không gian chật hẹp:

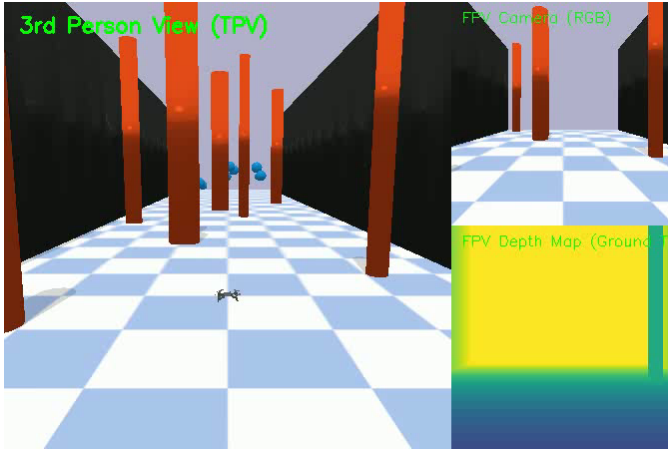


Fig. 3. Mô phỏng môi trường bay trên nền tảng PyBullet.

- **Cấu trúc đường đua:** Quãng đường bay dài 40 mét, bắt đầu tại vị trí $[0, 0, 1]$ và đích đến (Goal) tại $[0, 40, 1]$. Đường đua bị giới hạn bởi hai bức tường bao song song tạo thành một hành lang có chiều rộng 7.0 mét và chiều cao 2.5 mét.
- **Khu vực 1 - Trụ chướng ngại vật (Cylinders):** Kéo dài từ $y = 2$ m đến $y = 18$ m, bao gồm 9 trụ hình trụ đứng được rải ngẫu nhiên dọc theo đường đi. Các trụ có bán kính dao động từ 0.15 m đến 0.35 m và chiều cao 5.0 m, buộc UAV phải thể hiện khả năng lượn lách theo phương ngang (yaw/roll).
- **Khu vực 2 - Cầu bay lơ lửng (Floating Spheres):** Kéo dài từ $y = 20$ m đến $y = 38$ m, bao gồm 18 khối cầu chướng ngại vật lơ lửng trong không trung. Các khối cầu có bán kính ngẫu nhiên $[0.25, 0.4]$ m, được chia làm 6 hàng. Mỗi hàng chứa 3 quả cầu ở 3 mức độ cao khác

nhau (Thấp: 0.35 – 0.75 m; Trung bình: 0.9 – 1.25 m; Cao: 1.4 – 1.8 m). Thiết lập này là thử thách đặc biệt đánh giá khả năng lượn lách tổng hợp 3D của Custom PA-MPPI, bao gồm cả khả năng lấy độ cao và hạ độ cao đột ngột (pitch/thrust).

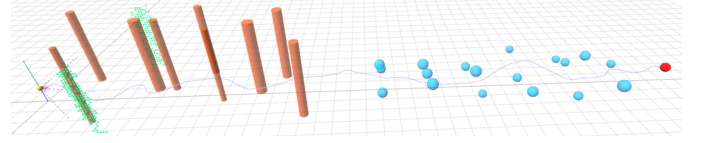


Fig. 4. Toàn cảnh đường bay 3D của UAV mô phỏng trên nền tảng thử nghiệm thẳng.

IV. CÁC CHỈ SỐ ĐÁNH GIÁ (EVALUATION METRICS)

Nhằm định lượng độ tin cậy và hiệu năng động lực học của hệ thống, chúng tôi áp dụng 5 chỉ số đo lường chính thức, được tính toán trung bình trên N lượt thử nghiệm:

- 1) **Tỷ lệ thành công (Success Rate - SR):** Phần trăm số chuyến bay UAV di chuyển thành công đến vùng không gian của mục tiêu (Goal) mà không xảy ra bất kỳ va chạm nào hoặc không bị hết thời gian (Timeout).

$$SR = \frac{N_{\text{success}}}{N_{\text{total}}} \times 100\% \quad (18)$$

- 2) **Tỷ lệ va chạm (Collision Rate - CR):** Phần trăm chuyến bay bị hủy do UAV xảy ra va chạm. Một chuyến bay được ghi nhận là va chạm (collision) khi khoảng cách từ tâm UAV đến bề mặt vật cản gần nhất nhỏ hơn ngưỡng vật lý 0.02 m (trùng khớp với bán kính thiết kế của CF2X), hoặc khi UAV mất độ cao và đâm xuống sàn (tọa độ $Z < 0.08$ m).

$$CR = \frac{N_{\text{collision}}}{N_{\text{total}}} \times 100\% \quad (19)$$

- 3) **Tiến độ quỹ đạo (Tracking Completion Rate - TCR):** Đánh giá mức độ tiến sát đến mục tiêu của UAV (tính bằng phần trăm khoảng cách ban đầu đã được thu hẹp) trong những chuyến bay thất bại do va chạm hoặc hết thời gian. Với d_{start} là khoảng cách ban đầu và $d_{\text{final}}^{(i)}$ là khoảng cách cuối cùng tới đích ở lượt thử nghiệm thứ i .

$$TCR = \frac{1}{N_{\text{total}}} \sum_{i=1}^{N_{\text{total}}} \max \left(0, \frac{d_{\text{start}} - d_{\text{final}}^{(i)}}{d_{\text{start}}} \right) \times 100\% \quad (20)$$

- 4) **Vận tốc trung bình (Mean Speed):** Trung bình cộng của độ lớn các vector vận tốc $\mathbf{v}_t$ xuyên suốt quỹ đạo bay T bước của các chuyến bay thành công, phản ánh khả năng duy trì động năng ở tốc độ cao.

$$\bar{v} = \frac{1}{T} \sum_{t=1}^T \|\mathbf{v}_t\|_2 \quad (21)$$

- 5) **Độ giật trung bình (Mean Jerk):** Đánh giá mức độ trơn tru (Smoothness) của tín hiệu điều khiển. Độ giật

thấp đồng nghĩa với việc quỹ đạo mượt mà, hạn chế hao mòn cơ khí và tiết kiệm năng lượng. Giả sử $\mathbf{a}_t$ là gia tốc tại thời điểm t :

$$\text{Jerk} = \frac{1}{T} \sum_{t=1}^T \left\| \frac{\mathbf{a}_t - \mathbf{a}_{t-1}}{\Delta t} \right\|_2 \quad (22)$$

V. KẾT QUẢ THỰC HIỆN

A. Đánh giá mô hình nhận thức (FastViT + Custom DPT)

Dựa trên bộ dữ liệu hỗn hợp (Mixed Dataset) gồm 78,861 mẫu được trình bày ở phần trước, mô hình ODANetFast được huấn luyện trong 10 epoch. Quá trình huấn luyện áp dụng chiến lược học tập phân rã Cosine (Cosine Annealing) với tốc độ học (Learning Rate) khác biệt cho phần thân (Backbone) và phần đầu giải mã (Head) nhằm bảo toàn các đặc trưng trích xuất sẵn có của ImageNet. Hình 5 trình bày biểu đồ thể hiện sự hội tụ của mô hình qua các epoch.

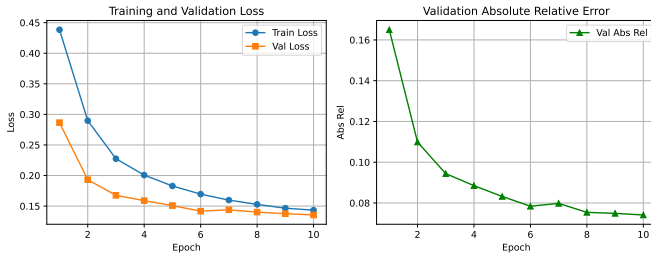


Fig. 5. Biểu đồ Loss và Absolute Relative Error (Abs Rel) của mô hình ODANetFast (FastViT-T12 + DPT) qua 10 epoch huấn luyện.

Như quan sát được trên Hình 5, hàm suy hao (Loss) trên tập kiểm định giảm ổn định và tiệm cận mức 0.135 ở epoch cuối. Tương tự, chỉ số sai số tương đối tuyệt đối (Absolute Relative Error - Abs Rel) cũng liên tục được cải thiện và đạt mức 0.0741. Xuyên suốt quá trình huấn luyện, mức tiêu thụ bộ nhớ GPU (Max VRAM) cao nhất ghi nhận ở khoảng 4.39 GB.

Để đánh giá định tính, Hình 6 thể hiện kết quả dự đoán của mô hình trên một mẫu dữ liệu trong tập kiểm định (Val Split) ở epoch thứ 10. Mặc dù là mẫu ảnh có cấu trúc vật cản phức tạp, mô hình ODANetFast đã tách biệt được các ranh giới độ sâu của vật thể. Kết quả dự đoán bám khá sát nhãn tham chiếu (Pseudo-depth label) được tạo ra bởi mô hình Depth Anything V2 (DA2), cho thấy mô hình có khả năng thu nhận các đặc trưng độ sâu từ mô hình giáo viên (Teacher Model).

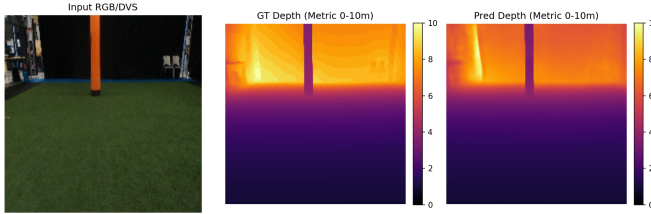


Fig. 6. So sánh định tính ở Epoch 10: Ảnh gốc RGB (trái), Nhãn Pseudo-depth do DA2 tạo ra (giữa) và Bản đồ độ sâu dự đoán bởi ODANetFast (phải).

B. Kết quả mô phỏng trên PyBullet

Nhằm đánh giá một cách toàn diện khả năng của bộ điều khiển Custom PA-MPPI, một đợt thử nghiệm quy mô lớn (Batch Simulation) gồm 500 chuyến bay đã được thực hiện trong môi trường mô phỏng 3D giả lập (đã mô tả tại mục III.C). Đợt thử nghiệm này so sánh hai cấu hình đầu vào nhận thức, cùng sử dụng chung bộ điều khiển Custom PA-MPPI:

- 1) **Ground Truth (GT) Depth (Baseline):** Sử dụng dữ liệu chiều sâu tuyệt đối thu thập trực tiếp từ engine PyBullet. Cấu hình này đóng vai trò như một hệ thống được trang bị cảm biến đo khoảng cách chuyên dụng (Depth Camera hoặc LiDAR) với độ chính xác hoàn hảo và không có nhiễu, dùng làm thước đo chuẩn (upper-bound) cho năng lực của bộ điều khiển.
- 2) **ODANetFast (Tích hợp End-to-End):** Sử dụng bản đồ độ sâu nội suy từ ảnh RGB đơn sắc (Monocular RGB) bởi mạng FastViT + Custom DPT.

Mặc dù cấu hình GT Depth (Depth Camera) mang lại độ chính xác cao, việc trang bị cảm biến chiều sâu chuyên dụng trên các vi thiết bị bay (Micro-UAVs) trong thực tế gặp nhiều rào cản. Thứ nhất, các dòng Depth Camera (như Intel RealSense) hoặc LiDAR đòi hỏi tải trọng (payload) vượt quá khả năng đáp ứng của các UAV cỡ nhỏ như Crazyflie 2.X (nền tảng này có tổng khối lượng chỉ 27g). Thứ hai, các cảm biến chủ động phát tia hồng ngoại (IR) tiêu thụ nhiều điện năng, làm giảm thời gian bay vốn đã ngắn của vi UAV, đồng thời dễ bị nhiễu sóng khi hoạt động ngoài trời. Do đó, phương pháp sử dụng một camera RGB thụ động, siêu nhẹ (vài gram) kết hợp mạng nơ-ron nội suy chiều sâu (ODANetFast) là giải pháp thực tiễn và thiết yếu cho các nền tảng bay siêu nhỏ.

Các chỉ số được sử dụng để đánh giá hiệu năng bay bao gồm: Tỷ lệ hoàn thành mục tiêu (Success Rate), Tỷ lệ va chạm (Collision Rate), Tỷ lệ hoàn thành quỹ đạo (Tracking Completion Rate - TCR), Vận tốc trung bình (Mean Speed) và Độ giật trung bình (Mean Jerk). Đặc biệt, một sự kiện va chạm (Collision) được ghi nhận một cách nghiêm ngặt khi khoảng cách từ tâm UAV đến vỏ vật cản nhỏ hơn 0.05 m ($d < 0.05$ m). Do bán kính sải cánh vật lý của UAV là $r = 0.06$ m, việc thiết lập ngưỡng 0.05 m tương đương với việc cho phép độ trễ sai số (Tolerance margin) là 1 cm. Điều này giúp loại trừ những sai số mép viền đồ họa (Polygon edges) trong engine PyBullet, đảm bảo hệ thống Benchmark chỉ đánh giá thất bại đối với các vụ đâm đụng vật lý thực sự (Hard Crashes) khi cánh quạt đã lún sâu vào vật cản, chứ không phạt những pha lướt mép (Grazing). Kết quả thống kê từ 500 chuyến bay được trình bày chi tiết trong Bảng IV.

Qua Bảng IV và Hình 7, hệ thống baseline sử dụng Ground Truth Depth đạt hiệu năng định tuyến tốt với 92.4% tỷ lệ bay tới đích an toàn và tiến độ hoàn thành quỹ đạo (TCR) đạt mức $96.9 \pm 13.2\%$. UAV giữ được khoảng cách an toàn với vật cản ($d_{\min} = 0.323 \pm 0.138$ m). Kết quả này cho thấy tính hiệu quả của thuật toán Custom PA-MPPI khi thiết lập hàm chi phí bất đối xứng (Asymmetric Progress) và sử dụng đám mây điểm cuộn (Rolling Point Cloud) trong không gian 3D phức tạp.

TABLE IV
Kết quả đánh giá hiệu năng bay tránh vật cản qua 500 lượt mô phỏng.

Metric	Ground Truth	ODANetFast	Depth Anything V2
Trials	500	500	500
Success Rate $\uparrow$	92.4%	24.2%	17.2%
Collision Rate $\downarrow$	7.2%	69.4%	76.4%
Timeout Rate $\downarrow$	0.4%	6.4%	6.4%
Tracking Completion Rate $\uparrow$	96.9 $\pm$ 13.2%	56.5 $\pm$ 36.1%	53.6 $\pm$ 38.2%
Mean Speed [m/s] $\uparrow$	2.16 $\pm$ 0.05	2.16 $\pm$ 0.06	2.26 $\pm$ 0.12
Flight Time [s] $\downarrow$	20.73 $\pm$ 0.89	21.69 $\pm$ 1.40	22.91 $\pm$ 0.93
Total Dist [m] $\downarrow$	44.75 $\pm$ 1.75	46.84 $\pm$ 2.85	51.63 $\pm$ 3.14
$d_{\min}$ [m] $\uparrow$	0.323 $\pm$ 0.138	0.122 $\pm$ 0.105	0.198 $\pm$ 0.594
Smoothness $\downarrow$	1.54 $\pm$ 0.04	1.53 $\pm$ 0.05	1.50 $\pm$ 0.05
Mean Jerk $\downarrow$	0.0104 $\pm$ 0.0008	0.0115 $\pm$ 0.0011	0.0109 $\pm$ 0.0010

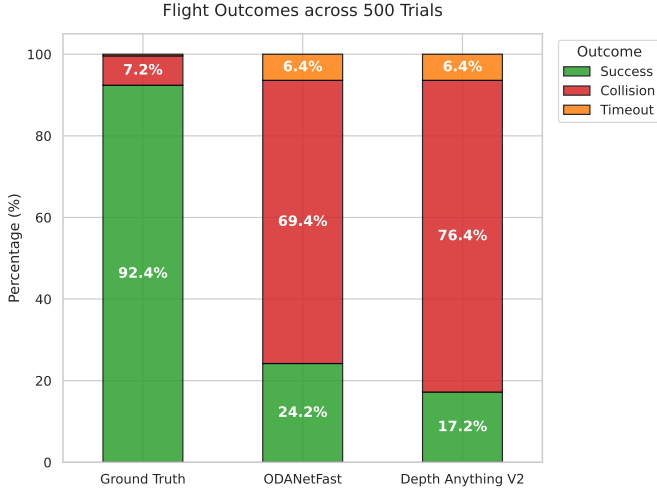


Fig. 7. Phân phối tỷ lệ kết quả (Thành công, Va chạm, Quá thời gian) qua 500 lượt mô phỏng.

Tuy nhiên, khi chuyển sang cấu hình tích hợp mô hình nhận thức thị giác máy tính ODANetFast (End-to-End), tỷ lệ thành công giảm xuống mức 24.2% và tỷ lệ va chạm tăng vọt lên 69.4%. Ngoài ra, khoảng cách an toàn tối thiểu sụt giảm mạnh chỉ còn 0.122 ± 0.105 m, và bản thân mạng suy luận chiều sâu trong suốt đợt thử nghiệm ghi nhận sai số tuyệt đối trung bình $MAE = 1.280 \pm 0.190$ m. Nguyên nhân chủ yếu xuất phát từ *khoảng cách miền* (Domain Gap) giữa môi trường ảnh tĩnh dùng để huấn luyện và dữ liệu động lực học vòng kín (Closed-loop) thực tế. Đặc biệt, việc sử dụng các camera có góc nhìn dọc hạn chế (FOV 69°) đã tạo ra vùng điểm mù lớn, khiến quỹ đạo bị chệch hướng dễ dẫn đến va chạm nếu vật cản vô tình trượt khỏi khung hình. Thêm vào đó, UAV sử dụng ODANetFast thường phải đánh vông quãng đường dài hơn (46.84 m so với 44.75 m) và mất nhiều thời gian hơn để xử lý tình huống.

Đặc biệt, kết quả thử nghiệm cho thấy mô hình giáo viên Depth Anything V2 (DA2) đạt hiệu năng thấp nhất với tỷ lệ thành công chỉ 17.2% và tỷ lệ va chạm cao nhất (76.4%), mặc dù đây là mô hình lớn với năng lực trích xuất đặc trưng tốt (xem Hình 11). Nguyên nhân cốt lõi là do DA2 nguyên bản là một mô hình dự đoán chiều sâu tương đối (Relative Depth Estimation). Dữ liệu đầu ra của nó (dải giá trị 0 – 255) chỉ đại diện cho nghịch đảo khoảng cách (disparity) và thiếu

thang đo vật lý cố định. Mặc dù trong quá trình mô phỏng, hệ thống đã nỗ lực ánh xạ thủ công dải giá trị này về thang đo tuyệt đối thông qua phép biến đổi tuyến tính $D_{\text{metric}} = 10.0 \times (1.0 - D_{\text{relative}}/255.0)$, phép biến đổi ngây thơ (naive scaling) này hoàn toàn phá vỡ cấu trúc không gian hình học. Thực tế, chiều sâu tỷ lệ nghịch (non-linear inverse) với giá trị ảnh relative, chứ không phải tỷ lệ tuyến tính. Sự sai lệch hình học (geometric distortion) này khi chiếu lên không gian 3D đã làm sai lệch hoàn toàn tọa độ của vật thể trong Rolling Sparse Point Cloud. Hậu quả là, bộ điều khiển PA-MPPI (vốn yêu cầu thông tin mét để tính biên giới va chạm 0.45 m) đã ước lượng sai khoảng cách và điều khiển UAV đâm trực diện vào chướng ngại vật.

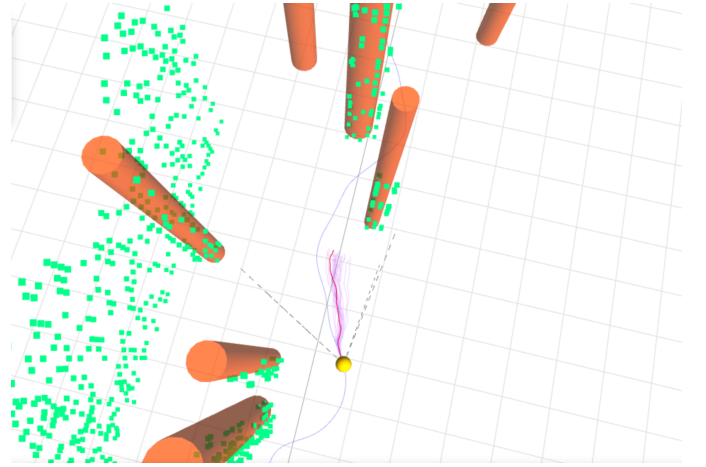


Fig. 8. Trực quan hóa thuật toán Custom PA-MPPI sinh ra chùm quỹ đạo (tím) để tìm ra đường lách tối ưu (đỏ). Các điểm màu xanh lá thể hiện Đám mây điểm vật cản thưa (Sparse Point Cloud) lưu trữ trong Rolling Buffer, và nét đứt màu xám biểu diễn Trường nhìn (FOV) của camera.

Đáng chú ý, khi xét ở khía cạnh ổn định bay, ODANetFast đạt hiệu suất tương đương baseline. Cả hai hệ thống đều duy trì được vận tốc bay ổn định quanh mức 2.16 ± 0.06 m/s. Chỉ số độ mượt hành động (Smoothness) của hệ thống thị giác máy tính là 1.53 ± 0.05 so với 1.54 ± 0.04 của baseline, và độ giật (Mean Jerk) không có sự chênh lệch lớn (0.0115 so với 0.0104). Điều này chỉ ra rằng, bản thân bộ điều khiển PA-MPPI hoạt động ổn định; điểm nghẽn (bottleneck) hiện tại chủ yếu nằm ở độ chính xác của mạng chiều sâu trong không gian động khép kín.

C. Đánh giá hiệu năng phần cứng (Hardware Benchmark)

Bên cạnh các chỉ số định tuyến, hiệu suất thời gian thực cũng được đo lường trực tiếp trên phần cứng GPU RTX 3060. Các cấu hình mạng học sâu đều được tối ưu hóa sang định dạng ONNX (CUDA Execution Provider) [9] để đảm bảo tính công bằng. Bảng V trình bày mức độ tiêu thụ tài nguyên của toàn bộ luồng quy hoạch vòng kín, bao gồm: suy luận chiều sâu, trích xuất Point Cloud và chạy bộ điều khiển JAX PA-MPPI.

Đánh giá phần cứng (Hardware Benchmark) được thực hiện độc lập, lấy trung bình qua 100 vòng lặp điều khiển (Control Loops) chính thức sau khi đã loại trừ 20 vòng lặp khởi động

TABLE V

So sánh hiệu năng điện toán chi tiết của toàn bộ hệ thống vòng kín trên RTX 3060.

Metric	Ground Truth Depth	ODANetFast	Depth Anything V2
Total Latency [ms] ↓	16.75 ± 0.88	21.28 ± 0.72	22.46 ± 0.47
Depth Inf. Time [ms] ↓	0.00 ± 0.00	4.87 ± 0.38	6.12 ± 0.14
Mapping Time [ms] ↓	2.65 ± 0.66	2.31 ± 0.26	2.20 ± 0.21
MPPI Control Time [ms] ↓	14.11 ± 0.39	14.10 ± 0.33	14.14 ± 0.34
Total Loop Freq [Hz] ↑	59.8 ± 3.1	47.0 ± 1.4	44.5 ± 0.9
Peak VRAM [MB] ↓	803.1 ± 0.0	951.1 ± 0.0	1079.1 ± 0.0
Power Usage [W] ↓	120.1 ± 34.9	131.2 ± 11.2	129.4 ± 4.5
Energy/Loop [J] ↓	2.02 ± 0.63	2.79 ± 0.26	2.91 ± 0.13

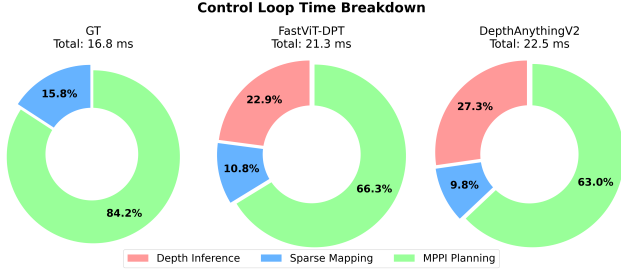


Fig. 9. Phân bố phần trăm thời gian xử lý của 3 thành phần (Depth Inference, Sparse Mapping, MPPI Planning) trong một chu kỳ điều khiển.

(Warm-up) để đảm bảo các tiến trình tối ưu trên CUDA/JAX đã hoạt động ổn định. Kết quả phân rã thời gian xử lý ở Bảng V và Hình 9 cho thấy điểm nghẽn (bottleneck) điện toán lớn nhất nằm ở bộ điều khiển PA-MPPI. Quá trình tính toán tối ưu MPPI tiêu thụ ổn định ~ 14.1 ms bất kể nguồn dữ liệu độ sâu là gì, chiếm tỷ trọng cao nhất (63% – 84%) trong tổng thời lượng của một chu kỳ do phải thực thi nội suy cho 10,000 quỹ đạo song song. Trong khi đó, mô hình nhận thức ODANetFast và Depth Anything V2 (chạy qua ONNX) chỉ tiêu tốn lần lượt 4.87 ms và 6.12 ms cho mỗi lần suy luận độ sâu từ ảnh RGB. Quá trình ánh xạ điểm thưa (Sparse Mapping) rất tối ưu, chỉ chiếm khoảng 2.20 – 2.65 ms. Nhờ đó, tần số điều khiển tổng quát (Total Loop Freq) của ODANetFast vẫn được duy trì ở mức cao 47.0 ± 1.4 Hz. Hơn nữa, với kiến trúc mạng nhẹ, mức VRAM đỉnh của hệ thống với ODANetFast chỉ là 951.1 MB, chứng minh tính khả thi cao khi ứng dụng các cấu trúc Pipeline tương tự trên nền tảng Edge AI bị giới hạn điện năng.

VI. BÀN LUẬN VÀ HẠN CHẾ

Mặc dù hệ thống đã thể hiện tính khả thi trong việc kết hợp mạng nhận thức thị giác và điều khiển tối ưu, báo cáo vẫn ghi nhận một số hạn chế nhất định cần được xem xét và khắc phục:

- **Khoảng cách miền do giới hạn của mô phỏng:** Quá trình huấn luyện và đánh giá hiện tại được thực hiện hoàn toàn trên engine PyBullet. Dù cung cấp nền tảng động lực học tốt, trình kết xuất đồ họa (renderer) của PyBullet vẫn tương đối đơn giản. Dữ liệu RGB sinh ra chưa phản ánh đầy đủ các tác động vật lý ở môi trường thực như hiện tượng nhòe chuyển động (motion blur) khi bay ở

tốc độ cao, nhiễu quang học (lens flare), hay sự rung lắc camera liên tục do luồng khí động học của cánh quạt. Điều này có thể làm suy giảm độ chính xác của mạng ODANetFast khi triển khai thực tế (Sim-to-Real gap).

- **Cấu hình UAV chưa phù hợp thực tế triển khai:** Hệ thống hiện mô phỏng nền tảng vi thiết bị bay Crazyflie 2.X (CF2X) với khối lượng cất cánh chỉ 27g. Trong thực tế, một nền tảng vi mô như vậy không có đủ sức nâng (payload) để mang theo một cụm máy tính Edge AI (như NVIDIA Jetson hay Raspberry Pi) nhằm xử lý toàn bộ pipeline vòng kín tại biên. Do đó, cấu hình hiện tại mang tính chất đánh giá thuật toán là chính. Để bay tự hành thực tế, hệ thống đòi hỏi một khung máy bay (frame) lớn hơn, ví dụ như drone 5-inch (tải trọng 500g – 1kg).
- **Bỏ qua yếu tố động lực học của vật cản:** Các kịch bản thử nghiệm hiện tại chỉ bao gồm các chương ngại vật đứng yên với hình học cơ bản. Trong môi trường thực tiễn, UAV thường xuyên phải đối mặt với các vật thể có cấu trúc phức tạp (cành cây mỏng, dây điện) và đặc biệt là các chương ngại vật di động (động vật, phương tiện, hoặc con người). Do cấu trúc Rolling Sparse Point Cloud hiện chưa được tích hợp khả năng lưu trữ thông tin về vận tốc tương đối của vật thể, thuật toán PA-MPPI chưa thể dự báo điểm va chạm trong tương lai của các vật thể động.
- **Giới hạn năng lực của mô hình nhận thức:** Mặc dù ODANetFast (với hạt nhân FastViT-T12) có ưu điểm về độ nhỏ gọn và tốc độ suy luận, nhưng do lượng tham số quá ít (~ 9.3 triệu), năng lực biểu diễn của mô hình bị giới hạn lớn. Khi đưa vào mô phỏng vòng kín, mô hình sinh ra sai số dự đoán chiều sâu cao ($MAE = 1.280 \pm 0.190$ m), khiến chất lượng đám mây điểm suy giảm và kéo theo sự sụt giảm nghiêm trọng của Tỷ lệ thành công (SR sụt giảm mạnh từ 92.4% của baseline xuống chỉ còn 24.2%).

VII. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Báo cáo này đã trình bày một luồng xử lý (pipeline) toàn trình cho hệ thống tránh vật cản UAV, kết hợp giữa mạng nhận thức và điều khiển. Về mặt quy hoạch quỹ đạo, bộ điều khiển Custom PA-MPPI đã cho thấy khả năng định tuyến thông qua kết quả Benchmark Ground Truth (92.4% Success). Tuy nhiên, nhận thức từ ảnh đơn sắc bằng mô hình hạng nhẹ ODANetFast cần được cải thiện độ chính xác hơn nữa.

Để hoàn thiện hệ thống, hướng phát triển tương lai sẽ tập trung vào các khía cạnh sau:

- Nghiên cứu triển khai các mô hình dự đoán độ sâu (Depth Estimation) lớn hơn có năng lực biểu diễn cao hơn, hoặc chuyển đổi sang sử dụng cảm biến chiều sâu chuyên biệt (Depth Camera) khi rào cản tải trọng phần cứng được nới lỏng.
- Nhằm đối phó với các vật thể động di chuyển nhanh, hệ thống sẽ hướng tới việc tích hợp cảm biến sự kiện (Event Camera) giúp nắm bắt thay đổi không gian với độ trễ cực thấp.
- Thay vì PyBullet, quá trình mô phỏng sẽ được thực hiện trên các nền tảng giả lập đồ họa chân thực hơn (như

NVIDIA Isaac Sim hoặc Unreal Engine/AirSim) nhằm loại bỏ hoàn toàn khoảng cách miền (Sim-to-Real gap).

- Để khắc phục điểm yếu của mạng nhận thức đơn ảnh (Monocular Perception) trước hiện tượng nhòe chuyển động (motion blur) và mất mát bối cảnh khi góc bề lái quá gắt (do giới hạn FOV), hệ thống sẽ hướng tới việc tích hợp cơ chế bộ nhớ thời gian (Temporal Memory) ngay tại tầng trích xuất đặc trưng (Feature Layer) của mạng nơ-ron. Các giải pháp như mạng LSTM/GRU hoặc cơ chế Chú ý ngữ cảnh hình học (Geometry Context Attention) [10] sẽ giúp ODANetFast tự động liên kết các khung hình trong quá khứ, từ đó triệt tiêu hiện tượng nhiễu nháy (flickering) và cung cấp đầu vào độ sâu ổn định tuyệt đối cho bộ đệm Rolling Buffer khi UAV bay ở tốc độ cao.

REFERENCES

- [1] G. Williams, A. Aldrich, and E. A. Theodorou, "Model predictive path integral control: From theory to parallel computation," *Journal of Guidance, Control, and Dynamics*, vol. 40, no. 2, pp. 344–357, 2017.
- [2] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018. [Online]. Available: <http://github.com/google/jax>
- [3] P. K. A. Vasu, J. Gabriel, J. Zhu, O. Tuzel, and A. Ranjan, "Fastvit: A fast hybrid vision transformer using structural reparameterization," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2023, pp. 5785–5795.
- [4] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision transformers for dense prediction," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 12 179–12 188.
- [5] E. Coumans and Y. Bai, "Pybullet, a python module for physics simulation for games, robotics and machine learning," <http://pybullet.org>, 2016–2021.
- [6] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao, "Depth anything v2," *Advances in Neural Information Processing Systems*, vol. 37, pp. 21 875–21 911, 2024.
- [7] Y. Zhai, R. Reiter, and D. Scaramuzza, "Pa-mppi: Perception-aware model predictive path integral control for quadrotor navigation in unknown environments," *IEEE Robotics and Automation Letters*, 2026.
- [8] J. Panerati, H. Zheng, S. Zhou, J. Xu, A. Prorok, and A. P. Schoellig, "Learning to fly—a gym environment with pybullet physics for reinforcement learning of multi-agent quadcopter control," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 7512–7519.
- [9] O. R. developers, "Onnx runtime," <https://onnxruntime.ai/>, 2021, version: 1.27.0.
- [10] L.-Z. Chen, J. Gao, Y. Chen, K. L. Cheng, Y. Sun, L. Hu, N. Xue, X. Zhu, Y. Shen, Y. Yao, and Y. Xu, "Geometric context transformer for streaming 3d reconstruction," *arXiv preprint arXiv:2604.14141*, 2026.

APPENDIX A PHỤ LỤC HÌNH ẢNH

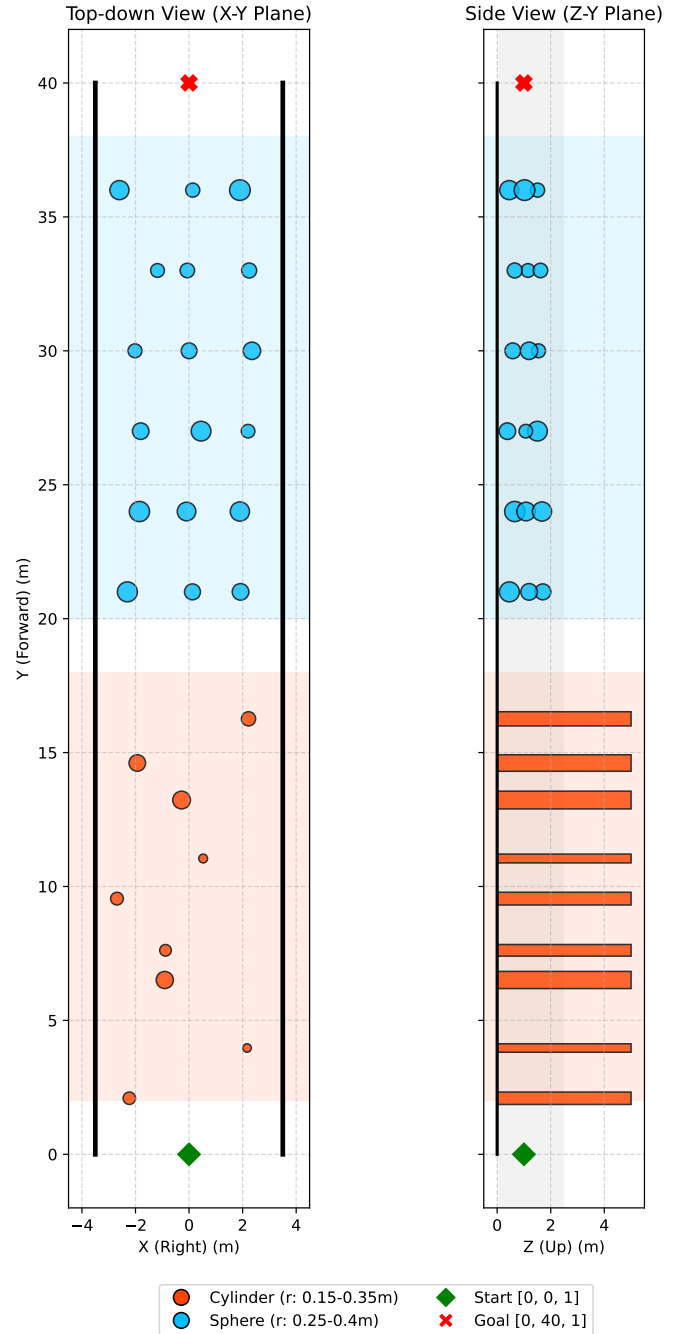


Fig. 10. Sơ đồ môi trường thử nghiệm đường bay thẳng (Straight Racing Track). Hình ảnh bao gồm hai góc nhìn: từ trên xuống (Top-down view, bên trái) và từ mặt bên (Side view, bên phải), thể hiện rõ các vùng chú trọng ngại vật hình trụ và hình cầu dọc theo quỹ đạo 40m.

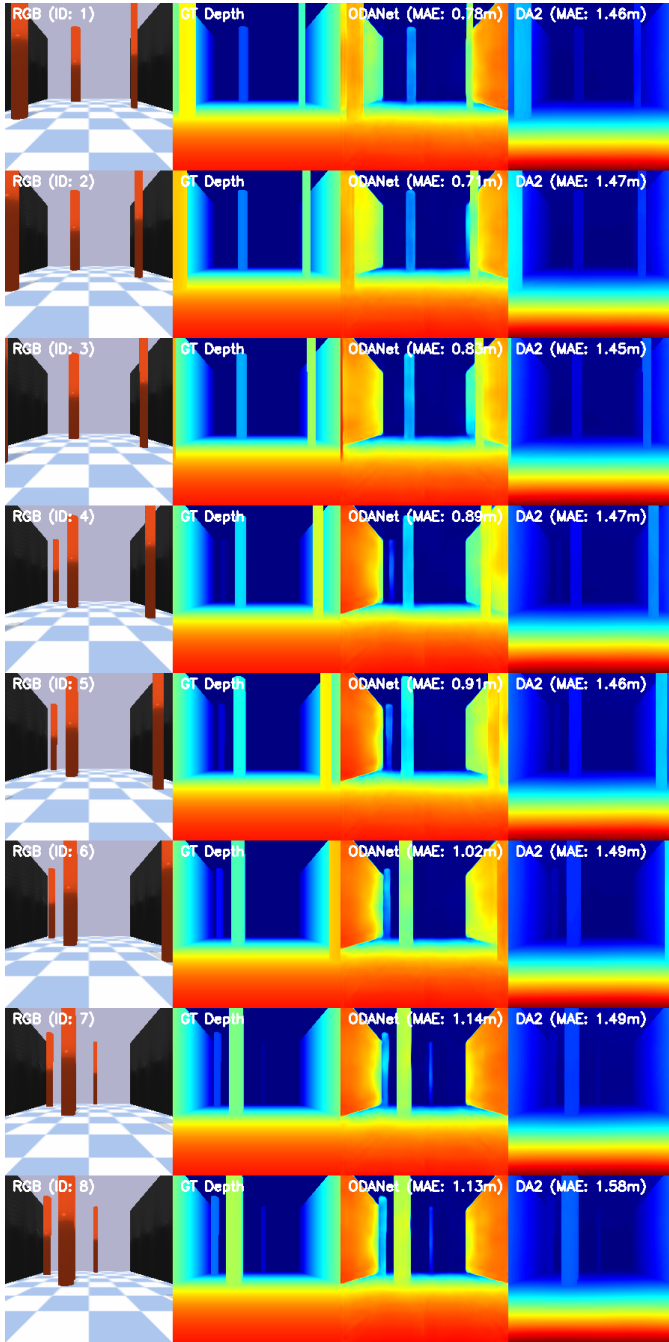


Fig. 11. So sánh định tính bản đồ độ sâu trong mô phỏng vòng kín: RGB gốc (cột 1), Ground Truth Depth (cột 2), ODANetFast (cột 3, MAE ~ 0.78 - 1.14 m), và Depth Anything V2 (cột 4, MAE ~ 1.45 - 1.58 m). Các chỉ số sai số tuyệt đối trung bình (MAE) được tính toán trên các điểm ảnh hợp lệ ($D > 0$) và được giới hạn ở ngưỡng cắt xa (Far Clipping Plane) 10.0 mét.